Author Contributions Checklist

This form documents the artifacts associated with *Smoothly Tilted MCMC Importance Sampling for Small Tail Probabilities* and describes how to reproduce its computational findings.

Part 1: Data

- ☒ This paper does not involve analysis of external data (i.e., the only data are generated by the authors via simulation in their code).
- ☒ The authors have legitimate permission to use and distribute all data used in this manuscript.

Abstract

The data are author-generated fixed-label permutation-test scenarios. The 160 near-threshold scenarios comprise 80 GWAS-like additive-score examples and 80 HEP-like Poisson-count examples. A further 160 GWAS-like scenarios cover ratios far below and above the significance threshold. Exact p-values are computed by dynamic programming. Archived run-level MCMC-IS and SAMC checkpoint records, bootstrap summaries, and repeated-chain OBM summaries support the reported computational results.

Availability

- ☒ Data are publicly available.
- ☐ Data cannot be made publicly available.

Publicly available data

- ☒ Data are available online at: https://github.com/NoamChowers/smoothly_tilted_mcmcis
- ☒ Data are available as part of the paper's supplementary material.
- ☐ Data are publicly available by request.
- ☐ Data are available through some other mechanism.

Description

File formats

- ☒ CSV or other plain text: CSV, JSON, and JSON Lines.
- ☐ Software-specific binary format.
- ☒ Standardized binary format: NumPy `.npy` arrays.
- ☒ Other: gzip compression for large JSON Lines files.

Data dictionary

- ☒ Provided by authors in `data/data_dictionary.csv` and `data/README.md`.
- ☐ Data files are self-describing.
- ☐ Available at another URL.

Additional information

All 320 scenario arrays can be independently regenerated from documented seeds and checked for exact equality using `make verify-data`. SHA-256 digests for the submission contents are in `checksums.sha256`.

Part 2: Code

Abstract

The Python code implements smoothly tilted self-normalized MCMC importance sampling, a hard-step variant, SAMC, exact dynamic-programming validation, data generation, the full simulation designs, repeated-chain OBM calibration, and article table/figure generation. Unit and reproducibility tests cover estimator behavior, checkpoint equivalence, exact scenario regeneration, record inventory, configuration selection, and headline table values.

Description

Code formats

- ☒ Script files
  - ☒ Python
- ☒ Package
  - ☒ Python
- ☐ Reproducible report
- ☐ Shell script
- ☒ Other: Makefile orchestration.

Supporting software requirements

Primary software used Python 3.13.0. The code supports Python 3.10 and later, but the supplied reference environment uses Python 3.13.0.

Libraries and dependencies Exact direct dependency versions are listed in `requirements-reproducibility.txt`; a Conda specification is in `environment.yml`. The reference versions are NumPy 2.1.3, Matplotlib 3.9.2, Pandas 2.2.3, SciPy 1.15.1, pytest 9.0.2, and setuptools 80.9.0.

Supporting system/hardware requirements

The reference outputs were generated on macOS arm64. Linux and Windows are also supported. No GPU or network connection is required after installation. Post-processing and smoke tests fit comfortably on a standard desktop. Full simulation reruns require long-running compute but no special hardware.

Parallelization used

- ☐ No parallel code used.
- ☒ Multi-core parallelization on a single machine/node.
  - Number of cores used: 6 for the original production experiments.
- ☐ Multi-machine/multi-node parallelization.

License

- ☒ MIT License.
- ☐ BSD.
- ☐ GPL v3.0.
- ☐ Creative Commons.
- ☐ Other.

Additional information

The development repository is https://github.com/NoamChowers/smoothly_tilted_mcmcis. The supplementary ZIP and `checksums.sha256` are the authoritative peer-review snapshot. A release tag or permanent archive DOI should be recorded before publication.

Part 3: Reproducibility workflow

Scope

The provided workflow reproduces:

- ☒ Numbers supplied in the article's computational results table and computational diagnostics.
- ☒ The computational methods presented in the paper.
- ☐ All tables and figures in the paper.
- ☒ Selected tables and figures: every computational table and figure. The remaining table and statements are analytic/theoretical and are derived in the manuscript rather than from data analysis.

Workflow

Location

- ☒ As part of the paper's supplementary material.
- ☒ In https://github.com/NoamChowers/smoothly_tilted_mcmcis.
- ☐ Other.

Formats

- ☒ Single master code file: `workflow/reproduce.py`.
- ☐ Wrapper shell scripts.
- ☐ Self-contained literate programming document.
- ☒ Text documentation: `README.md` and `workflow/README.md`.
- ☒ Makefile.
- ☒ Other: installable Python package and compressed run-level records.

Instructions

Create a Python 3.13 environment, install `requirements-reproducibility.txt`, install the local package with `python -m pip install -e .`, and run `make all`. This verifies every generated scenario, runs the tests, and writes all computational article outputs to `output/reproduced/`. Run `make smoke` for short fresh estimator trajectories. See `README.md` for full commands and the article-output map.

Expected run-time

Approximate time needed on a standard desktop:

- ☐ $<$ 1 minute.
- ☒ 1–10 minutes for the complete reviewer workflow from archived run-level records, after dependency installation.
- ☐ 10–60 minutes.
- ☐ 1–8 hours.
- ☒ $>$ 8 hours for a clean rerun of all Monte Carlo trajectories (`make full` and `make full-obm`).
- ☐ Not feasible on a desktop machine.

Additional information

The archived records allow fast independent checking of every reported computational output. The full modes are supplied to establish complete provenance, while the smoke modes confirm fresh end-to-end method execution. PDF byte hashes may differ across font/PDF backends; the numeric CSV and JSON outputs are the comparison targets.

Notes

One-million-resample BCa confidence bounds are included from the article's seeded production bootstrap. The workflow recomputes their underlying point estimates from run-level records and reuses the archived bounds, avoiding a different bootstrap realization while keeping the exact reported intervals.